

INTERVIEW & PRACTICE GUIDE

React.js Interview Guide & Projects

Two-part reference: 71 commonly asked React.js interview questions with clear answers, plus 12 hands-on practice projects with complete, working solutions — the kind of small builds used in coding rounds and take-home assessments.

PART A

71 Interview Q&A

PART B

12 Mini Projects

STACK

React 18+ (Hooks)

Table of Contents

PART A — INTERVIEW QUESTIONS

01	React Fundamentals	8 questions
02	Components, Props & JSX	6 questions
03	State & Core Hooks	8 questions
04	Advanced Hooks	8 questions
05	Lifecycle & Class Components	5 questions
06	Forms & Events	5 questions
07	Lists, Keys & Conditional Rendering	4 questions
08	Context API & State Management	5 questions
09	Performance Optimization	6 questions
10	Routing (React Router)	5 questions
11	Error Handling & Testing	5 questions
12	Comparison & Tricky Questions	6 questions

PART B — PRACTICE PROJECTS

01	Counter App	useState
02	Todo List (CRUD)	arrays & keys
03	Digital Clock	useEffect
04	Stopwatch	useRef + interval
05	Accordion / FAQ	conditional render
06	Tabs Component	derived state
07	Star Rating	hover state
08	Traffic Light	state machine
09	Debounced Search	custom hook
10	useFetch + User List	data fetching
11	Theme Toggle	Context API

.....

Q1. What is React and why is it popular?

React is a JavaScript library (created by Facebook/Meta) for building user interfaces out of reusable, composable components. It's popular because of its declarative programming model (you describe *what* the UI should look like for a given state, not the step-by-step DOM manipulation), the Virtual DOM for efficient updates, a huge ecosystem, and a component-based architecture that scales well across large teams and codebases.

Q2. What is the Virtual DOM and how does reconciliation work?

The Virtual DOM is a lightweight in-memory JavaScript representation of the real DOM. When state changes, React builds a new Virtual DOM tree, **diffs** it against the previous one (a process called reconciliation), and computes the minimal set of real DOM mutations needed — then applies only those, instead of re-rendering the entire page. This makes updates far cheaper than direct DOM manipulation for every change.

Q3. What is JSX?

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like markup directly inside JavaScript. It's not understood by browsers directly — a compiler (Babel/SWC) transforms it into plain `React.createElement()` calls (or the newer automatic JSX runtime) before it ships to the browser.

Q4. What is the difference between a React element and a React component?

An **element** is a plain, immutable JavaScript object describing what should appear on screen (the output of `React.createElement` or JSX) — cheap to create. A **component** is a function (or class) that accepts props and *returns* elements; it's the reusable blueprint, while an element is just one snapshot of its output.

Q5. Is React a library or a framework?

React is a **library**, specifically focused on the view layer (rendering UI from state). Unlike a full framework, it doesn't prescribe routing, data fetching, or state management out of the box — those are chosen separately (React Router, Redux/Zustand, etc.), or you use a framework like Next.js built on top of React that adds those opinions.

Q6. What does `ReactDOM.createRoot` do?

It's the React 18+ entry point that creates a root container attached to a real DOM node and enables concurrent rendering features. Calling `.render(<App />)` on that root mounts the component tree into the page. It replaced the legacy `ReactDOM.render()` API, which only supported the older, synchronous rendering mode.

Q7. What is React Fiber?

Fiber is React's internal reconciliation engine (since React 16), which reworked rendering into an incremental, interruptible unit-of-work model. It allows React to pause, prioritize, and resume rendering work — the foundation that makes concurrent features like `useTransition`, `Suspense`, and time-slicing possible.

Q8. What are React Server Components, briefly?

Server Components render entirely on the server and send no JavaScript to the browser for that component, reducing bundle size and allowing direct backend access. They're a rendering model used by frameworks built on React (most notably Next.js's App Router) rather than a feature of a plain client-side React app created with Vite/CRA.

Q9. What is the difference between functional and class components?

	Functional Component	Class Component
Syntax	Plain function returning JSX	Extends <code>React.Component</code> , defines <code>render()</code>
State	Hooks (<code>useState</code> , etc.)	<code>this.state</code> / <code>this.setState()</code>
Lifecycle	<code>useEffect</code>	<code>componentDidMount</code> , etc.
Current status	Recommended standard	Legacy, still supported

Q10. What are props and can a component modify its own props?

Props ("properties") are read-only inputs passed from a parent component into a child, used to configure how the child renders or behaves. Props are **immutable** from the child's perspective — a component must never modify `this.props` or a destructured prop directly; if the value needs to change, that logic belongs in the parent, which then passes down new props.

Q11. What is "prop drilling" and how do you avoid it?

Prop drilling is passing a prop down through several intermediate components that don't use it themselves, just to reach a deeply nested child. It's avoided with the **Context API** (for moderate cases), a state management library like `Redux/Zustand`, or by restructuring components (composition, passing components as children instead of data).

Q12. What is the children prop?

A special prop automatically passed to a component containing whatever was nested between its opening and closing JSX tags, e.g. `<Card><p>Hello</p></Card>` gives the `Card` component a `children` prop equal to the `<p>` element. It enables flexible composition — wrapper/layout components that don't need to know what they'll render inside.

Q13. How do you type-check props in React?

With the `prop-types` package (runtime checks via `Component.propTypes = { ... }`) for plain JavaScript projects, or with **TypeScript** interfaces/types for compile-time checking, which is the more common modern approach since it catches mismatches before the code even runs.

Q14. What are React Fragments and why use them?

`<>...</>` (or `<React.Fragment>`) lets a component return multiple sibling elements without wrapping them in an extra DOM node (like a `<div>`), since a component can only return a single root element. This avoids unnecessary wrapper elements that could break CSS layout (e.g. flex/grid) or add meaningless nesting.

Q15. What is state, and how does `useState` work?

State is data local to a component that, when changed, triggers a re-render so the UI reflects the new value. `useState` returns a pair: the current value and a setter function to update it and schedule a re-render.

```
const [count, setCount] = useState(0);
```

Q16. Why does `useState`'s setter appear asynchronous / batched?

React batches multiple state updates that occur within the same event handler (and, since React 18, across most async boundaries too) into a single re-render for performance — rather than re-rendering after every individual `setState` call. This means reading the state variable immediately after calling its setter still shows the old value, since the update hasn't been applied yet.

Q17. What is the functional/callback form of a state updater, and when is it needed?

Passing a function to the setter (`setCount(prev => prev + 1)`) guarantees you're updating from the latest state, which matters when multiple updates happen in quick succession or depend on the previous value — using the captured variable directly (`setCount(count + 1)` called twice) can silently drop an update due to stale closures.

```
setCount((prev) => prev + 1);
```

Q18. What is `useEffect` and how does its dependency array control when it runs?

`useEffect` runs a side effect (data fetching, subscriptions, manual DOM work) after render. The dependency array controls re-execution: `[]` runs once after the first render, `[a, b]` re-runs whenever `a` or `b` changes, and omitting the array entirely runs the effect after every render.

```
useEffect(() => {  
  document.title = `Count: ${count}`;  
}, [count]);
```

Q19. What's the difference between `useEffect` and `useLayoutEffect`?

`useEffect` runs asynchronously *after* the browser paints the screen, so it doesn't block visual updates. `useLayoutEffect` runs synchronously *before* the browser paints, blocking the paint until it finishes — used when you need to measure or mutate the DOM (e.g. reading layout dimensions) before the user sees a visual flicker.

Q20. What is the cleanup function in `useEffect` for?

A function returned from the effect that React calls before the effect re-runs (dependencies changed) and when the component unmounts — used to cancel subscriptions, clear timers, or remove event listeners so they don't leak or fire against a stale/unmounted component.

```
useEffect(() => {  
  const id = setInterval(() => console.log('tick'), 1000);  
  return () => clearInterval(id); // cleanup  
}, []);
```

Q21. What are common `useEffect` mistakes?

- **Missing dependencies** — using a variable inside the effect without listing it, causing stale closures.
- **Infinite loops** — updating state inside the effect without a proper dependency array, so the effect re-runs on every render it causes.
- **Forgetting cleanup** — leaving timers/listeners/subscriptions running after unmount.
- **Using an effect for something derivable during render** — computed values often don't need an effect at all.

Q22. What's the difference between controlled and uncontrolled components?

A **controlled** component's value is driven entirely by React state (`value` + `onChange`), making state the single source of truth. An **uncontrolled** component keeps its own internal DOM state and is read on demand via a `ref` (e.g. `<input defaultValue="..." ref={inputRef} />`). Controlled components are generally preferred for validation and predictable state, while uncontrolled ones can be simpler for very basic forms.

Q23. What is `useRef` used for?

It creates a mutable object (`{ current: ... }`) that persists across renders **without** triggering a re-render when changed — used for accessing DOM nodes directly, or storing any mutable value (timer IDs, previous values) that shouldn't cause a re-render on update.

```
const inputRef = useRef(null);
const focusInput = () => inputRef.current.focus();
return <input ref={inputRef} />;
```

Q24. What is `useMemo` and when should you use it?

`useMemo` memoizes the **result** of an expensive computation, recalculating it only when its dependencies change instead of on every render. Use it for genuinely expensive calculations (large array processing, complex derived data) — overusing it on cheap computations adds overhead without benefit.

```
const sorted = useMemo(() => [...items].sort(compareFn), [items]);
```

Q25. What is `useCallback` and how does it differ from `useMemo`?

`useCallback` memoizes a **function reference** itself (not a computed value) so it doesn't get recreated on every render — important when passing callbacks to memoized child components (`React.memo`) or as effect dependencies, to prevent unnecessary re-renders/re-runs. `useMemo` memoizes a value; `useCallback(fn, deps)` is essentially `useMemo(() => fn, deps)`.

```
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);
```

Q26. When would you prefer `useReducer` over `useState`?

When state logic is complex — multiple sub-values that update together, or the next state depends on a specific action/type rather than a simple new value. `useReducer` centralizes that logic in one pure reducer function, making transitions easier to test and reason about compared to scattering several related `useState` calls.

```
function reducer(state, action) {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: return state;
  }
}

const [state, dispatch] = useReducer(reducer, { count: 0 });
```

Q27. What is useContext and how does it work?

It lets a component read a value from the nearest matching Context.Provider above it in the tree, without needing that value passed down explicitly through every intermediate component's props — the core mechanism for solving prop drilling.

Q28. What is a custom hook, and what are the rules for writing one?

A custom hook is simply a JavaScript function whose name starts with use and that calls other hooks internally, letting you extract and reuse stateful logic (e.g. a debounce hook, a fetch hook) across multiple components. It must follow the same Rules of Hooks as built-in hooks.

```
function useToggle(initial = false) {  
  const [value, setValue] = useState(initial);  
  const toggle = () => setValue((v) => !v);  
  return [value, toggle];  
}
```

Q29. What are the Rules of Hooks?

1. Only call hooks at the **top level** of a function component or custom hook — never inside loops, conditions, or nested functions.
 2. Only call hooks from **React functions** (components or custom hooks), not plain JavaScript functions.
- These rules exist because React relies on the *order* hooks are called in across renders to correctly associate each hook call with its internal state.

Q30. What are useTransition and useDeferredValue used for?

Both are React 18 concurrent-rendering hooks for keeping the UI responsive during expensive updates. useTransition marks a state update as low-priority ("non-urgent"), letting React interrupt it to handle more urgent updates (like typing) first. useDeferredValue gives you a "lagging" copy of a value that updates after more urgent renders have completed — useful for keeping a search input responsive while a large results list re-renders behind it.

Q31. What are the three main phases of a component's lifecycle?

Mounting (component is created and inserted into the DOM), **Updating** (re-renders due to new props/state), and **Unmounting** (component is removed from the DOM). Class components expose dedicated methods for each phase; function components handle all three through `useEffect` and its cleanup/dependency behavior.

Q32. How does `componentDidMount` map to hooks?

`useEffect(() => { ... }, [])` with an empty dependency array runs once after the initial render, equivalent to `componentDidMount`. Its cleanup function (the returned function) is equivalent to `componentWillUnmount`, and an effect with dependencies covers `componentDidUpdate`-like behavior.

Q33. What is `shouldComponentUpdate` / `PureComponent`?

`shouldComponentUpdate(nextProps, nextState)` lets a class component return `false` to skip a re-render when it decides the change doesn't matter — a manual performance optimization. `React.PureComponent` does this automatically via a shallow comparison of props and state. The function-component equivalent is wrapping the component with `React.memo`.

Q34. What is `getDerivedStateFromProps`?

A rarely-needed static class lifecycle method called right before every render (both mount and update) that lets a component update its state based on changes in props. It's intentionally restrictive (static, no access to `this`) to discourage overuse; most cases are better served by computing values directly during render instead of syncing them into state.

Q35. Why did the React community move from class components to hooks?

- Hooks let you reuse stateful logic via custom hooks, whereas classes needed awkward patterns like Higher-Order Components or render props.
- Related logic (e.g. subscribing and unsubscribing) can live together in one `useEffect` instead of being split across `componentDidMount`/`componentWillUnmount`.
- No need to reason about `this` binding for event handlers.
- Function components with hooks are generally more concise and easier to test.

Q36. How do you build a controlled form input?

Bind the input's `value` to a state variable and update that state in the `onChange` handler, so React state is always the source of truth for what's displayed.

```
const [name, setName] = useState('');  
<input value={name} onChange={(e) => setName(e.target.value)} />
```

Q37. How do you handle multiple form inputs with a single change handler?

Give each input a matching `name` attribute, store all fields in one state object, and use the input's `name` to update just that key with computed-property syntax.

```
const [form, setForm] = useState({ name: '', email: '' });  
  
const handleChange = (e) => {  
  const { name, value } = e.target;  
  setForm((prev) => ({ ...prev, [name]: value }));  
};
```

Q38. What is a SyntheticEvent in React?

A cross-browser wrapper object around the browser's native event that React passes to event handlers, normalizing event properties/behavior across different browsers. As of React 17+, synthetic events are attached at the root container rather than the document, though they still behave like native events for most practical purposes (e.g. `preventDefault()` works as expected).

Q39. How does event delegation work in React?

Rather than attaching a real DOM listener to every single element, React attaches one listener at the root of the rendered tree and uses event bubbling to determine which component's handler should run for a given event — more memory-efficient than binding thousands of individual native listeners.

Q40. How do you prevent a form's default submission or stop event bubbling?

```
const handleSubmit = (e) => {  
  e.preventDefault(); // stop the browser's default form submit/reload  
  e.stopPropagation(); // stop the event from bubbling to parent handlers  
  // ...submit logic  
};
```

07 Lists, Keys & Conditional Rendering

Q41. Why does React require a key prop when rendering lists?

Keys give React a stable identity for each list item across renders, so it can correctly match items between the old and new Virtual DOM trees during reconciliation — determining which items were added, removed, or reordered, instead of re-rendering the entire list from scratch.

Q42. Why is using the array index as a key discouraged?

If the list can be reordered, filtered, or have items inserted/removed, the index no longer reliably identifies the same logical item across renders — React can end up reusing the wrong DOM node/state for a different item (e.g. an input's typed text "sticking" to the wrong row). A stable, unique identifier from the data itself (like a database ID) should be used instead. Index keys are acceptable only for static lists that never reorder or change.

Q43. What are common patterns for conditional rendering in JSX?

```
{isLoggedIn && <Dashboard />}           {/* render or nothing */}
{isLoggedIn ? <Dashboard /> : <Login />}    {/* either/or */}
{status === 'loading' && <Spinner />}
{status === 'error' && <ErrorMessage />}
```

Q44. Can a component render nothing? How?

Yes — returning `null` (or `false`/`undefined`) from a component tells React to render nothing for that component, without throwing an error. This is commonly used for components that conditionally show/hide themselves based on props or state.

Q45. What problem does the Context API solve?

It provides a way to share values (theme, authenticated user, locale, etc.) across a component tree without manually passing props down through every intermediate level — solving prop drilling for state that many components across different nesting depths need to read.

Q46. What's a common performance pitfall with Context?

Every component that consumes a context re-renders whenever the context value changes, even if it only cares about part of that value. Putting frequently-changing state in one large context can cause widespread unnecessary re-renders; splitting contexts by concern, or memoizing the provided value, helps limit the blast radius.

Q47. What are the core building blocks of Redux?

- **Store** — the single source of truth holding the entire app state.
- **Actions** — plain objects describing "what happened" (`{ type: 'ADD_TODO', payload }`).
- **Reducers** — pure functions that take the current state and an action and return the new state.
- **Dispatch** — the only way to trigger a state change, by sending an action to the store.

Q48. When should you reach for Context vs. a library like Redux/Zustand?

Context is well suited for relatively static, infrequently-changing global values (theme, auth user, locale) shared across many components. For complex, frequently-updated application state with many interdependent pieces, time-travel debugging needs, or middleware (logging, async flows), a dedicated state management library scales better and typically renders more efficiently by design.

Q49. What is Redux Toolkit and why is it recommended over plain Redux?

Redux Toolkit (RTK) is the official, opinionated toolset for Redux that drastically reduces boilerplate: `createSlice` auto-generates action creators and reducers, it includes Immer under the hood (so you can write "mutating" update logic that's actually applied immutably), and it bundles good defaults (DevTools, thunk middleware) out of the box.

Q50. What does `React.memo` do?

It wraps a function component so React skips re-rendering it if its props haven't changed (using a shallow comparison by default), similar to `PureComponent` for class components. Useful for components that render often but with unchanged props — especially list items or expensive-to-render children.

```
const Row = React.memo(function Row({ item }) {  
  return <li>{item.name}</li>;  
});
```

Q51. How does code splitting work with `React.lazy` and `Suspense`?

`React.lazy` lets you import a component dynamically so it's only downloaded when actually needed, splitting it into a separate bundle chunk. Wrapping it in `<Suspense fallback={...}>` shows a fallback UI while that chunk loads.

```
const Settings = React.lazy(() => import('./Settings'));  
  
<Suspense fallback=<Spinner />>  
  <Settings />  
</Suspense>
```

Q52. What are common causes of unnecessary re-renders, and how do you avoid them?

- Creating new object/array/function literals inline as props on every render — breaks memoization comparisons; fix with `useMemo/useCallback`.
- Storing state too high in the tree, causing a large subtree to re-render for a small local change — move state closer to where it's used.
- Not memoizing pure child components with `React.memo` when their parent re-renders often.
- Using Context for rapidly-changing values consumed by many components.

Q53. What is list virtualization and when do you need it?

Virtualization (e.g. via `react-window` or `react-virtualized`) renders only the list items currently visible in the viewport (plus a small buffer), instead of every item in a potentially huge dataset. It's needed when rendering long lists (thousands of rows) where mounting every DOM node up front would be slow and memory-heavy.

Q54. How do you implement debouncing in a React component?

Delay acting on a rapidly-changing value (like a search input) until the user pauses, using a `setTimeout` inside a `useEffect` that resets on every keystroke via its cleanup function — commonly extracted into a reusable `useDebounce` custom hook (see Project 9 in Part B).

Q55. How do `React.memo`, `useMemo`, and `useCallback` work together?

`React.memo` only helps if the props passed to the memoized component are referentially stable across renders. If a parent passes a new inline function or object as a prop every render, memoization is defeated. `useCallback` stabilizes function props and `useMemo` stabilizes computed object/array props, so the memoized child can actually skip re-rendering when nothing meaningful changed.

10 Routing (React Router)

Q56. What is client-side routing and how does React Router enable it?

Client-side routing changes which component is displayed based on the URL **without** triggering a full page reload from the server — React Router intercepts navigation, updates the browser's history/URL via the History API, and renders the matching route's component.

Q57. What's the difference between `BrowserRouter` and `HashRouter`?

`BrowserRouter` uses the HTML5 History API for clean URLs (`/about`) but requires server configuration to serve `index.html` for every route (so direct loads/refreshes work). `HashRouter` stores the route in the URL fragment (`/#/about`), which never gets sent to the server, so it works on static hosts without any server-side rewrite configuration — at the cost of a less clean URL.

Q58. How do you read a dynamic route parameter?

```
<Route path="/users/:id" element={<UserProfile />} />

function UserProfile() {
  const { id } = useParams();
  return <p>User ID: {id}</p>;
}
```

Q59. What are nested routes and what does `<Outlet />` do?

Nested routes let a parent route render shared layout while a child route fills in a specific section based on the URL. `<Outlet />` is the placeholder inside the parent layout component where the matched child route's element gets rendered.

```
<Route path="/dashboard" element={<DashboardLayout />}>
  <Route path="stats" element={<Stats />} />
  <Route path="settings" element={<Settings />} />
</Route>

function DashboardLayout() {
  return (
    <div>
      <Sidebar />
      <Outlet />
    </div>
  );
}
```

Q60. How do you implement a protected/private route?

Wrap the protected element in a component that checks authentication state and either renders the intended element or redirects (e.g. via `<Navigate />`) to a login page.

```
function ProtectedRoute({ isAuthenticated, children }) {  
  if (!isAuthenticated) return <Navigate to="/login" replace />;  
  return children;  
}
```

11 Error Handling & Testing

Q61. What is an Error Boundary?

A class component that implements `static getDerivedStateFromError()` and/or `componentDidCatch()` to catch JavaScript errors thrown anywhere in its child component tree during rendering, and display a fallback UI instead of crashing the whole app. There is currently no hook-based equivalent, so error boundaries still require a class component (though the boundary itself is usually a small, reusable wrapper).

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  render() {
    if (this.state.hasError) return <h1>Something went wrong.</h1>;
    return this.props.children;
  }
}
```

Q62. Why don't error boundaries catch errors in event handlers or async code?

Error boundaries only catch errors thrown during rendering, in lifecycle methods, and in constructors of the tree below them — not inside event handlers, `setTimeout` callbacks, or async code, since those execute outside of React's render cycle. Those cases need a regular `try/catch` around the async/handler logic itself.

Q63. What is the testing philosophy behind React Testing Library?

"Test your components the way a user would use them" — querying the rendered output by visible text, labels, and roles rather than internal implementation details (component instance state, private methods). This makes tests more resilient to refactors, since they only break when actual user-facing behavior changes.

Q64. How do you mock an API call in a component test?

Common approaches: mock the module doing the fetching directly (`jest.mock('./api')`), stub the global `fetch`/`axios` client, or intercept network requests at the HTTP layer with a library like Mock Service Worker (MSW) so the component's real `fetch` logic runs against a fake server response.

Q65. What are the pros and cons of Jest snapshot testing?

Pros: fast to write, catches unintended markup/output changes automatically. **Cons:** snapshots can grow large and hard to review meaningfully, developers often blindly approve ("update snapshot") failing tests without checking if the change was intentional, and they don't verify actual user-facing behavior the way interaction-based tests do.

12 Comparison & Tricky Questions

Q66. What is the core difference between state and props?

	State	Props
Owned by	The component itself	Passed in from the parent
Mutability	Mutable (via its setter)	Read-only from the child's side
Purpose	Internal data that changes over time	Configuration/data passed down

Q67. Why can't hooks be called conditionally or inside loops?

React tracks hooks by the **order** they're called in on every render, not by name, to know which internal state slot each `useState/useEffect` call corresponds to. If a hook call is skipped conditionally on one render but not another, the order shifts and React associates the wrong stored state with the wrong hook — corrupting component state unpredictably.

Q68. Does the entire component function re-run on every re-render?

Yes — a re-render means React calls the function component again from top to bottom, recreating local variables, inline functions, and JSX. What's *preserved* across renders is only what's stored in Hooks (state via `useState/useReducer`, refs via `useRef`), which React keeps tied to that component instance outside the function body itself.

Q69. Controlled vs. uncontrolled components — which should you default to?

Default to **controlled** components for most forms: state stays in sync with the UI at all times, enabling real-time validation, conditional disabling, and predictable testing. Reach for uncontrolled only for simple, rarely-changing inputs (e.g. a file input, or a form you only read once on submit) where the overhead of controlling every keystroke isn't worth it.

Q70. What happens if you update state with the exact same value it already has?

React uses `Object.is` comparison to bail out of re-rendering if the new state value is identical to the current one (for primitives). For objects/arrays, this means calling the setter with the same reference (not just equivalent content) skips a re-render — which is why patterns like `setItems([...items])` (a new array reference) are needed to force React to notice a change, even if the values look the same.

Q71. Why might a `useEffect` that fetches data run twice in development?

In React 18 Strict Mode (development only), React intentionally mounts, unmounts, and remounts components once to help surface effects that aren't properly cleaned up — simulating what would happen if a component's state needed to be restored later. It does not happen in production builds; the fix is ensuring the effect's cleanup function properly cancels/ignores the first fetch (see the `cancelled` flag pattern in Project 10).

PART B

Practice Projects

12 small, self-contained React components covering the patterns most often asked in coding rounds and take-home assessments: state management, timers and effects, custom hooks, data fetching, and the Context API. Each includes the concepts it's testing and a complete, working solution using functional components and hooks.

Project 1 — Counter App

BEGINNER

Requirements: Display a number with buttons to increment, decrement, and reset it.

```
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div style={{ textAlign: 'center', fontFamily: 'sans-serif' }}>
      <h1>{count}</h1>
      <button onClick={() => setCount(count - 1)}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

Tests: useState, event handlers, re-rendering on state change.

P2 Todo List (CRUD)

Project 2 — Todo List

BEGINNER

Requirements: Add a task, mark it complete/incomplete, and delete it.

```
import { useState } from 'react';

export default function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState('');

  const addTodo = () => {
    if (!text.trim()) return;
    setTodos([...todos, { id: Date.now(), text, done: false }]);
    setText('');
  };

  const toggleTodo = (id) =>
    setTodos(todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));

  const deleteTodo = (id) =>
    setTodos(todos.filter((t) => t.id !== id));

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} placeholder="Add a task" />
      <button onClick={addTodo}>Add</button>
      <ul>
        {todos.map((todo) => (
          <li key={todo.id} style={{ textDecoration: todo.done ? 'line-through' : 'none' }}>
            <span onClick={() => toggleTodo(todo.id)}>{todo.text}</span>
            <button onClick={() => deleteTodo(todo.id)}>Delete</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Tests: array state immutability (map/filter/spread), controlled input, list rendering with keys.

P3 Digital Clock

Project 3 — Digital Clock

BEGINNER

Requirements: Display the current time, updating every second.

```
import { useState, useEffect } from 'react';

export default function DigitalClock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const timer = setInterval(() => setTime(new Date()), 1000);
    return () => clearInterval(timer);
  }, []);

  return <h1>{time.toLocaleTimeString()}</h1>;
}
```

Tests: useEffect with cleanup, setInterval as a side effect.

Project 4 — Stopwatch

INTERMEDIATE

Requirements: Start, pause, and reset a running timer accurate to the centisecond.

```
import { useState, useRef } from 'react';

export default function Stopwatch() {
  const [time, setTime] = useState(0);
  const [running, setRunning] = useState(false);
  const intervalRef = useRef(null);

  const start = () => {
    if (running) return;
    setRunning(true);
    intervalRef.current = setInterval(() => setTime((t) => t + 10), 10);
  };

  const pause = () => {
    clearInterval(intervalRef.current);
    setRunning(false);
  };

  const reset = () => {
    clearInterval(intervalRef.current);
    setRunning(false);
    setTime(0);
  };

  const format = (ms) => {
    const minutes = String(Math.floor(ms / 60000)).padStart(2, '0');
    const seconds = String(Math.floor((ms % 60000) / 1000)).padStart(2, '0');
    const millis = String(Math.floor((ms % 1000) / 10)).padStart(2, '0');
    return `${minutes}:${seconds}:${millis}`;
  };

  return (
    <div>
      <h1>{format(time)}</h1>
      <button onClick={start}>Start</button>
      <button onClick={pause}>Pause</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}
```

Tests: useRef for a mutable interval ID that shouldn't trigger re-renders, functional state updates.

Project 5 — Accordion / FAQ Toggle**BEGINNER**

Requirements: Clicking a question expands its answer; only one item is open at a time.

```
import { useState } from 'react';

const faqs = [
  { q: 'What is React?', a: 'A JavaScript library for building UIs.' },
  { q: 'What is JSX?', a: 'A syntax extension for writing HTML-like code in JS.' },
  { q: 'What is a hook?', a: 'A function that lets you use state in function components.' },
];

export default function Accordion() {
  const [openIndex, setOpenIndex] = useState(null);
  const toggle = (i) => setOpenIndex(openIndex === i ? null : i);

  return (
    <div>
      {faqs.map((item, i) => (
        <div key={i}>
          <div onClick={() => toggle(i)} style={{ cursor: 'pointer', fontWeight: 'bold' }}>
            {item.q} {openIndex === i ? '-' : '+'}
          </div>
          {openIndex === i && <p>{item.a}</p>}
        </div>
      ))}
    </div>
  );
}
```

Tests: conditional rendering, "single open item" state pattern.

P6 Tabs Component

Project 6 — Tabs Component

BEGINNER

Requirements: Clicking a tab label shows only that tab's content.

```
import { useState } from 'react';

const tabsData = [
  { label: 'Home', content: 'Welcome to the home tab.' },
  { label: 'Profile', content: 'This is your profile.' },
  { label: 'Settings', content: 'Adjust your settings here.' },
];

export default function Tabs() {
  const [active, setActive] = useState(0);

  return (
    <div>
      <div style={{ display: 'flex', gap: 8 }}>
        {tabsData.map((tab, i) => (
          <button
            key={i}
            onClick={() => setActive(i)}
            style={{ fontWeight: active === i ? 'bold' : 'normal' }}
          >
            {tab.label}
          </button>
        ))}
      </div>
      <p>{tabsData[active].content}</p>
    </div>
  );
}
```

Tests: deriving displayed content from an index in state, list-rendered controls.

Project 7 — Star Rating**INTERMEDIATE**

Requirements: Click a star to set the rating; preview the rating on hover.

```
import { useState } from 'react';

export default function StarRating({ totalStars = 5 }) {
  const [rating, setRating] = useState(0);
  const [hover, setHover] = useState(0);

  return (
    <div>
      {Array.from({ length: totalStars }, (_, i) => i + 1).map((star) => (
        <span
          key={star}
          onClick={() => setRating(star)}
          onMouseEnter={() => setHover(star)}
          onMouseLeave={() => setHover(0)}
          style={{
            cursor: 'pointer',
            fontSize: '2rem',
            color: star <= (hover || rating) ? 'gold' : 'lightgray',
          }}
        >
          ★
        </span>
      ))}
      <p>Rating: {rating} / {totalStars}</p>
    </div>
  );
}
```

Tests: two related pieces of state (committed vs. hover preview), array generation with `Array.from`.

Project 8 — Traffic Light

INTERMEDIATE

Requirements: Cycle red → green → yellow → red automatically, each for a different duration.

```
import { useState, useEffect } from 'react';

const sequence = [
  { color: 'red', duration: 3000 },
  { color: 'green', duration: 3000 },
  { color: 'yellow', duration: 1000 },
];

export default function TrafficLight() {
  const [index, setIndex] = useState(0);

  useEffect(() => {
    const timer = setTimeout(() => {
      setIndex((prev) => (prev + 1) % sequence.length);
    }, sequence[index].duration);
    return () => clearTimeout(timer);
  }, [index]);

  const current = sequence[index].color;

  return (
    <div style={{ display: 'flex', flexDirection: 'column', gap: 8, width: 60 }}>
      {[ 'red', 'yellow', 'green' ].map((c) => (
        <div
          key={c}
          style={{
            width: 60,
            height: 60,
            borderRadius: '50%',
            background: current === c ? c : '#333',
            opacity: current === c ? 1 : 0.3,
          }}
        />
      ))}
    </div>
  );
}
```

Tests: an effect that depends on state and re-schedules itself, cleanup preventing leaked timers, a simple state machine.

Project 9 — Debounced Search Bar

ADVANCED

Requirements: Filter a list as the user types, but only after they pause typing (avoids filtering/API calls on every keystroke).

```
import { useState, useEffect } from 'react';

const allItems = ['Apple', 'Banana', 'Cherry', 'Date', 'Grape', 'Mango', 'Orange', 'Peach'];

function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debounced;
}

export default function DebouncedSearch() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 400);

  const results = allItems.filter((item) =>
    item.toLowerCase().includes(debouncedQuery.toLowerCase())
  );

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} placeholder="Search fruits" />
      <ul>
        {results.map((item) => <li key={item}>{item}</li>)}
      </ul>
    </div>
  );
}
```

Tests: writing a reusable custom hook, debounce via useEffect cleanup, separating "raw" input state from the "settled" value used for filtering.

Project 10 — Custom useFetch Hook + User List

ADVANCED

Requirements: Fetch data from an API and handle loading and error states cleanly, in a reusable way.

```
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    fetch(url)
      .then((res) => res.json())
      .then((json) => { if (!cancelled) { setData(json); setLoading(false); } })
      .catch((err) => { if (!cancelled) { setError(err); setLoading(false); } });
    return () => { cancelled = true; };
  }, [url]);

  return { data, loading, error };
}

export default function UserList() {
  const { data, loading, error } = useFetch('https://jsonplaceholder.typicode.com/users');

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Something went wrong.</p>;

  return (
    <ul>
      {data.map((user) => <li key={user.id}>{user.name}</li>)}
    </ul>
  );
}
```

Tests: a data-fetching custom hook, race-condition guarding with a cancelled flag, modeling loading/error/success as explicit states.

Project 11 — Theme Toggle (Context API)

INTERMEDIATE

Requirements: Toggle between light/dark theme, readable by any component without prop drilling.

```
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext();

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => setTheme((t) => (t === 'light' ? 'dark' : 'light'));

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

function useTheme() {
  return useContext(ThemeContext);
}

function ThemedBox() {
  const { theme, toggleTheme } = useTheme();
  return (
    <div style={{
      background: theme === 'light' ? '#fff' : '#111',
      color: theme === 'light' ? '#111' : '#fff',
      padding: 20,
    }}>
      <p>Current theme: {theme}</p>
      <button onClick={toggleTheme}>Toggle Theme</button>
    </div>
  );
}

export default function App() {
  return (
    <ThemeProvider>
      <ThemedBox />
    </ThemeProvider>
  );
}
```

Tests: Context API (createContext/Provider/useContext), wrapping context access in a custom hook, avoiding prop drilling.

P12 Pagination Component

Project 12 — Pagination Component

INTERMEDIATE

Requirements: Show a fixed page size of items with Prev/Next controls and disable buttons at the boundaries.

```
import { useState } from 'react';

export default function Pagination({ items, pageSize = 5 }) {
  const [page, setPage] = useState(1);
  const totalPages = Math.ceil(items.length / pageSize);
  const start = (page - 1) * pageSize;
  const currentItems = items.slice(start, start + pageSize);

  return (
    <div>
      <ul>{currentItems.map((item, i) => <li key={start + i}>{item}</li>)}</ul>
      <div>
        <button disabled={page === 1} onClick={() => setPage(page - 1)}>Prev</button>
        <span> Page {page} of {totalPages} </span>
        <button disabled={page === totalPages} onClick={() => setPage(page + 1)}>Next</button>
      </div>
    </div>
  );
}
```

Tests: deriving a visible slice from state + array data, disabling controls at boundary conditions.